ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 6
ШАБЛОНЫ DJANGO И ПРОЕКТНАЯ РАЗРАБОТКА

ЦЕЛЬ РАБОТЫ

Изучить систему шаблонов Django: базовый шаблон и наследование, включаемые шаблоны, передачу данных через контекст, условные конструкции и циклы, фильтры шаблонов, именованные URL-маршруты с пространствами имён и подключение статических файлов.

Перевести веб-интерфейс индивидуального проекта, созданный в ПР5, с формирования HTML-кода в Python-функциях на систему шаблонов Django, сохранив предметную область, JSON-хранилище и работоспособность существующего кода проекта.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 3
  2.1. Подготовка к работе 3
  2.2. Продолжение индивидуального проекта 3
  2.3. Проверка исходного состояния проекта 4
  2.4. Контрольная точка Git 5
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 6
  3.1. HTML-страница и Django 6
  3.2. Шаблоны Django 7
  3.3. Функция `render()` 8
  3.4. Контекст шаблона 8
  3.5. Выражения `{{ ... }}` 9
  3.6. Условия в шаблонах 9
  3.7. Циклы в шаблонах 10
  3.8. Наследование шаблонов 11
  3.9. Подключение частей шаблона через include 12
  3.10. Именованные URL и пространства имён 12

3.11. Фильтры шаблонов 13
3.12. Статические файлы 13
3.13. Архитектура веб-части Roomly после ПР6 14
3.14. Что должно измениться в проекте 14
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 15
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 16
5.1. Общая постановка задания 16
5.2. Создание структуры шаблонов 16
5.3. Создание базового шаблона 18
5.4. Создание главной страницы 19
5.5. Перевод homepage на render() и судьба функции page() 20
5.6. Перенос страниц помещений на шаблоны 21
5.6.1. Список помещений и функция render() 21
5.6.2. Карточка помещения и {% include %} 24
5.6.3. Страница отдельного помещения и обработка 404 26
5.7. Условное отображение данных 29
5.8. Перенос страниц бронирований на шаблоны 30
5.8.1. Список бронирований 30
5.8.2. Статус бронирования и {% include %} 31
5.8.3. Маршруты бронирований: проверка вместо изменений 31
5.8.4. Страница отдельного бронирования 32
5.8.5. Окончательный вид bookings/views.py и проверка 34
5.9. Общая навигация сайта 36
5.10. Фильтры шаблонов 37
5.11. Статические файлы проекта 38
5.11.1. Структура и подключение статических файлов 38
5.11.2. Собственный CSS проекта 39
5.11.3. Подключение JavaScript 40
5.11.4. Оформление карточек и статуса собственными стилями 41
5.12. Обновление README.md 42
5.13. Проверка и фиксация результата 46
5.13.1. Проверка запуска и всех страниц 46
5.13.2. Проверка перехода на шаблоны 47
5.13.3. Проверка оформления шаблонов 47
5.13.4. Проверка проекта 48
5.13.5. Фиксация результата в Git 48
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 49
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 50
7.1. Чек-лист выполнения материалов Яндекс Практикума 51
7.2. Чек-лист выполнения индивидуального проекта 51
7.3. Что необходимо показать при защите 52
7.4. Критерии оценки 53
7.5. Требования к отчету 53
8. КОНТРОЛЬНЫЕ ВОПРОСЫ 54
9. ИТОГ 56

1. ЗАДАЧИ РАБОТЫ

В ходе практической работы необходимо:

- изучить возможности шаблонов Django для формирования HTML-страниц;
- освоить разделение HTML-документа на базовый шаблон и отдельные блоки;
- научиться передавать данные из view-функций в шаблоны через контекст;
- использовать в шаблонах условные конструкции и циклы;

- организовать переходы между страницами с помощью именованных URL-маршрутов;
- освоить пространства имён URL-маршрутов Django-приложений;
- применять фильтры шаблонов для форматирования и обработки отображаемых данных;
- подключать к Django-шаблонам статические файлы: CSS, изображения и JavaScript;
- перенести существующие HTML-страницы проекта из ПР5 на систему шаблонов Django;
- сохранить работу с предметной областью индивидуального проекта на основе объектов `Room`, `User` и `Booking`;
- оформить результаты работы в Git и обновить `README.md`.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

2.1. Подготовка к работе

Перед началом работы:

- войти в личный аккаунт Яндекс, используемый для обучения;
- открыть курс Яндекс Практикума «Бэкенд-разработчик»;
- перейти к разделу 04 «Основы веб-разработки»;
- проверить состояние индивидуального проекта после ПР5;
- убедиться, что проект запускается, а изменения ПР5 зафиксированы в Git;
- ознакомиться с целью, задачами и требованиями настоящей практической работы.

ПР6 является логическим продолжением ПР5: переход к веб-разработке на основе шаблонов выполняется в том же репозитории индивидуального проекта. Создавать новый репозиторий не требуется.

2.2. Продолжение индивидуального проекта

Работа выполняется на основе индивидуального проекта, разработанного в ПР1–ПР5. Новый репозиторий для ПР6 создавать не требуется. В ПР6 необходимо продолжить развитие Django-проекта, созданного в ПР5.

Для примера используется проект `Roomly` – система бронирования помещений.

К началу работы проект должен иметь следующую структуру:

```
room_booking/
├── README.md
├── requirements.txt
├── main.py
├── storage.py
├── utils.py
│
```

```
├── models/
│   ├── __init__.py
│   ├── rooms.py
│   ├── users.py
│   └── bookings.py
│
├── data/
│   ├── rooms.json
│   ├── users.json
│   └── bookings.json
│
├── tests/
│   ├── test_rooms.py
│   ├── test_users.py
│   └── test_bookings.py
│
├── manage.py
│
├── roomly/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── homepage/
├── rooms/
└── bookings/
```

Примечание. Код консольной версии из ПР1–ПР3 удалять не следует. Он сохраняется как предыдущая реализация предметной области. В ПР6 изменяется именно веб-часть Django-приложения.

2.3. Проверка исходного состояния проекта

Перед началом работы необходимо убедиться, что проект из ПР5 запускается.

Активировать виртуальное окружение:

```
venv\Scripts\activate
```

Проверить Django:

```
python -m django --version
```

Проверить проект:

```
python manage.py check
```

Запустить сервер:

```
python manage.py runserver
```

Проверить страницы:

```
http://127.0.0.1:8000/
http://127.0.0.1:8000/rooms/
http://127.0.0.1:8000/rooms/1/
http://127.0.0.1:8000/bookings/
http://127.0.0.1:8000/bookings/1/
```

Если в проекте используются другие идентификаторы помещений и бронирований, необходимо использовать существующие значения.

До начала изменений необходимо убедиться, что:

- главная страница открывается;
- список помещений отображается;
- страница отдельного помещения открывается;
- список бронирований отображается;
- страница отдельного бронирования открывается;
- переходы между страницами работают;
- данные загружаются из JSON;
- существующие тесты проходят.

2.4. Контрольная точка Git

Перед началом доработки необходимо создать контрольную точку состояния проекта. Проверить состояние репозитория:

```
git status
```

Если в проекте остались незакоммиченные изменения, сначала завершите работу из ПР5 и сохраните корректное состояние проекта отдельным коммитом:

```
git add .
git commit -m "PR6: начало перехода на шаблоны Django"
```

Каждый последующий коммит, относящийся к ПР6, должен начинаться с метки PR6:. Например:

```
PR6: add base template
PR6: convert room pages to templates
PR6: add static files
```

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1. HTML-страница и Django

В ПР5 HTML-страницы формировались непосредственно в Python-коде. Например, каркас страницы собирала функция `page()` в файле `homepage/views.py`:

```
def page(title, content):
    return f"""<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta    name="viewport"    content="width=device-width,    initial-
scale=1">
    <title>{title}</title>
    <link   rel="stylesheet"   href="https://cdn.jsdelivr.net/npm/boot-
strap@5.3.3/dist/css/bootstrap.min.css">
</head>
<body>
    <nav class="nav">
        <a class="nav-link" href="/">Главная</a>
        <a class="nav-link" href="/rooms/">Помещения</a>
        <a class="nav-link" href="/bookings/">Бронирования</a>
    </nav>
    <main class="container">{content}</main>
</body>
</html>"""
```

view-функция формировала содержимое страницы и передавала его в `HttpResponse`:

```
def rooms(request):
    items = ""
    for room in load_rooms():
        items += f'<li class="list-group-item">{room.name} – вместимость
{room.capacity} мест</li>'
    content = f"""
    <h1>Помещения</h1>
    <ul class="list-group">{items}</ul>
    """
    return HttpResponse(page("Roomly – помещения", content))
```

Такой подход позволяет получить работающую страницу, но при дальнейшем развитии приложения приводит к смешиванию разных задач:

- обработка HTTP-запроса;
- получение данных;
- формирование HTML;

- оформление страницы;
- навигация.

В ПР6 HTML необходимо отделить от Python-кода. Вместо формирования HTML внутри `view` используется шаблон Django.

Общая схема становится следующей (рис. 1):

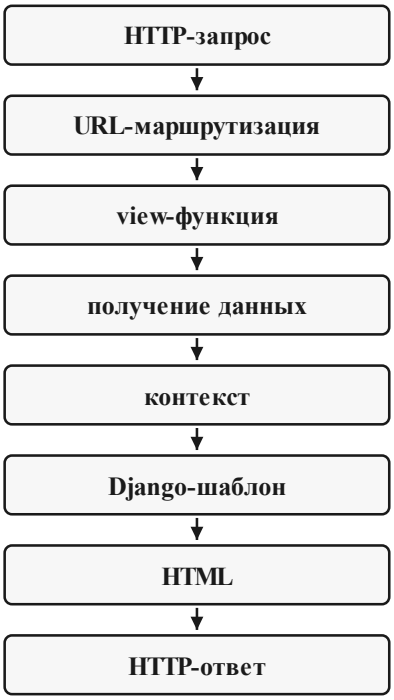


Рисунок 1 – Обработка запроса с использованием шаблона Django

3.2. Шаблоны Django

Шаблон Django – HTML-документ, содержащий специальные конструкции языка шаблонов Django. Шаблон отвечает за представление данных. Например:

```
<h1>{{ room.name }}</h1>
<p>Вместимость: {{ room.capacity }}</p>
```

Здесь `{{ room.name }}` означает получение значения `name` объекта `room`. Сам объект передаётся из view-функции. Например:

```
def room_detail(request, room_id):
    rooms_list = load_rooms()
    room = find_room_by_id(rooms_list, room_id)

    context = {
        "room": room,
    }

    return render(request, "rooms/room_detail.html", context)
```

Таким образом, Python-код не формирует HTML-строку. Он подготавливает данные для страницы.

3.3. Функция `render()`

Для формирования ответа на основе шаблона используется функция `render()`:

```
from django.shortcuts import render
```

Общий вид:

```
return render(request, "имя_шаблона.html", context)
```

где:

- `request` – текущий HTTP-запрос;
- второй аргумент – путь к шаблону;
- `context` – словарь данных, передаваемых шаблону.

Например:

```
def rooms(request):
    rooms_list = load_rooms()

    context = {
        "rooms": rooms_list,
    }

    return render(request, "rooms/room_list.html", context)
```

В шаблоне:

```
{% for room in rooms %}
    <h2>{{ room.name }}</h2>
    <p>Вместимость: {{ room.capacity }}</p>
{% endfor %}
```

Получается разделение ответственности:

Компонент	Ответственность
`URLconf`	определяет адрес страницы
`view`	получает и подготавливает данные
`context`	передаёт данные шаблону
`template`	формирует HTML
`CSS`	отвечает за оформление
`JavaScript`	реализует клиентское поведение

3.4. Контекст шаблона

Контекст – словарь, содержащий данные, которые должны быть доступны в шаблоне. Например:

```
context = {
    "rooms": rooms_list,
    "page_title": "Помещения Roomly",
}
```

В шаблоне эти значения доступны по соответствующим именам:

```
<h1>{{ page_title }}</h1>

{% for room in rooms %}
    <p>{{ room.name }}</p>
{% endfor %}
```

Важно соблюдать соответствие имён. Если во `view` используется:

```
context = {
    "rooms": rooms_list,
}
```

то в шаблоне необходимо обращаться к:

```
{{ rooms }}
```

а не к:

```
{{ rooms_list }}
```

если такая переменная отдельно в контекст не передавалась.

3.5. Выражения {{ ... }}

Для вывода значения в шаблоне используются двойные фигурные скобки:

```
{{ variable }}
```

Например:

```
{{ room.name }}
{{ room.capacity }}
{{ booking.status }}
```

Можно обращаться к атрибутам объектов, например `{{ room.name }}`, а также к элементам структур данных. В ПР6 необходимо использовать шаблоны для отображения объектов предметной области: `Room, User, Booking.`

При этом предметная логика продолжает находиться в Python-коде. Шаблон не должен превращаться в место реализации бизнес-логики.

3.6. Условия в шаблонах

Для условного отображения используется конструкция:

```
{% if условие %}
    ...
{% else %}
    ...
{% endif %}
```

Например:

```
{% if booking.is_cancelled %}
    <p>Бронирование отменено</p>
{% else %}
    <p>Бронирование активно</p>
{% endif %}
```

В Roomly условные конструкции необходимо использовать для отображения различных состояний данных.

Например:

- активное или отменённое бронирование;
- наличие или отсутствие помещений;
- наличие данных о пользователе;
- доступность помещения.

3.7. Циклы в шаблонах

Для перебора коллекции используется:

```
{% for объект in коллекция %}
    ...
{% endfor %}
```

Например:

```
{% for room in rooms %}
    <article>
        <h2>{{ room.name }}</h2>
        <p>Вместимость: {{ room.capacity }}</p>
    </article>
{% endfor %}
```

Для пустой коллекции можно использовать empty:

```
{% for room in rooms %}
    <article>
        <h2>{{ room.name }}</h2>
    </article>
{% empty %}
    <p>Помещения не найдены.</p>
{% endfor %}
```

Это позволяет обрабатывать ситуацию, когда данные отсутствуют, непосредственно в шаблоне.

3.8. Наследование шаблонов

Если каждая HTML-страница содержит одинаковые элементы:

- DOCTYPE;
- <head>;
- подключение CSS;
- меню;
- контейнер страницы;
- подвал

Копировать этот код в каждый файл не следует. Для общей структуры используется базовый шаблон. Например:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Roomly{% endblock %}</title>
</head>
<body>

    <nav>
        ...
    </nav>

    <main>
        {% block content %}
        {% endblock %}
    </main>

</body>
</html>
```

Другой шаблон наследует эту структуру:

```
{% extends "base.html" %}

{% block title %}
    Помещения
{% endblock %}

{% block content %}
    <h1>Помещения</h1>
{% endblock %}
```

Такой подход позволяет изменять общую структуру сайта в одном месте.

3.9. Подключение частей шаблона через include

Повторяющиеся элементы страницы можно вынести в отдельные шаблоны. Например, карточка помещения:

```
<article class="room-card">
    <h2>{{ room.name }}</h2>
    <p>Вместимость: {{ room.capacity }}</p>
</article>
```

Сохранить её можно как отдельный шаблон:

```
rooms/
└── templates/
    └── rooms/
        └── includes/
            └── room_card.html
```

В основном шаблоне:

```
{% for room in rooms %}
    {% include "rooms/includes/room_card.html" %}
{% endfor %}
```

Это позволяет избежать дублирования HTML-кода.

3.10. Именованные URL и пространства имён

В ПР5 адреса страниц указывались в HTML непосредственно:

```
<a href="/rooms/">Помещения</a>
```

В ПР6 необходимо перейти к именованным маршрутам. В `rooms/urls.py`:

```
app_name = "rooms"

urlpatterns = [
    path("", views.rooms, name="list"),
    path("<int:room_id>/", views.room_detail, name="detail"),
]
```

Теперь в шаблоне используется:

```
<a href="{% url 'rooms:list' %}">
    Помещения
</a>
```

Для конкретного помещения:

```
<a href="{% url 'rooms:detail' room.id %}">
    Подробнее
</a>
```

Преимущество такого подхода заключается в том, что HTML не зависит непосредственно от конкретного URL. Если маршрут изменится:

```
path("all/", views.rooms, name="list")
```

ссылка:

```
{% url 'rooms:list' %}
```

останется прежней.

3.11. Фильтры шаблонов

Фильтры позволяют изменять представление значения непосредственно в шаблоне. Например:

```
{{ rooms|length }}
```

выводит количество помещений.

Для значения, которое может отсутствовать, можно использовать:

```
{{ room.name|default:"Без названия" }}
```

Для даты:

```
{{ booking.booking_date|date:"d.m.Y" }}
```

Фильтры следует использовать для представления данных, а не для переноса бизнес-логики из Python в шаблон.

3.12. Статические файлы

CSS, изображения и JavaScript не следует помещать непосредственно в HTML-шаблон. Для статических файлов Django используется механизм `static`. В шаблоне необходимо загрузить библиотеку:

```
{% load static %}
```

После этого файл подключается через:

```
{% static "homepage/css/style.css" %}
```

Например:

```
<link rel="stylesheet"
      href="{% static 'homepage/css/style.css' %}">
```

Изображение:

```
<img src="{% static 'homepage/img/logo.png' %}"
     alt="Roomly">
```

JavaScript:

```
<script src="{% static 'homepage/js/main.js' %}"></script>
```

В ПР6 собственный CSS-файл проекта является обязательным. `Bootstrap`, использовавшийся в ПР5, можно сохранить, однако он не заменяет собственные статические файлы проекта.

3.13. Архитектура веб-части Roomly после ПР6

После выполнения работы веб-часть проекта должна перейти от схемы, применявшейся в ПР5, к шаблонной архитектуре (рис. 2).

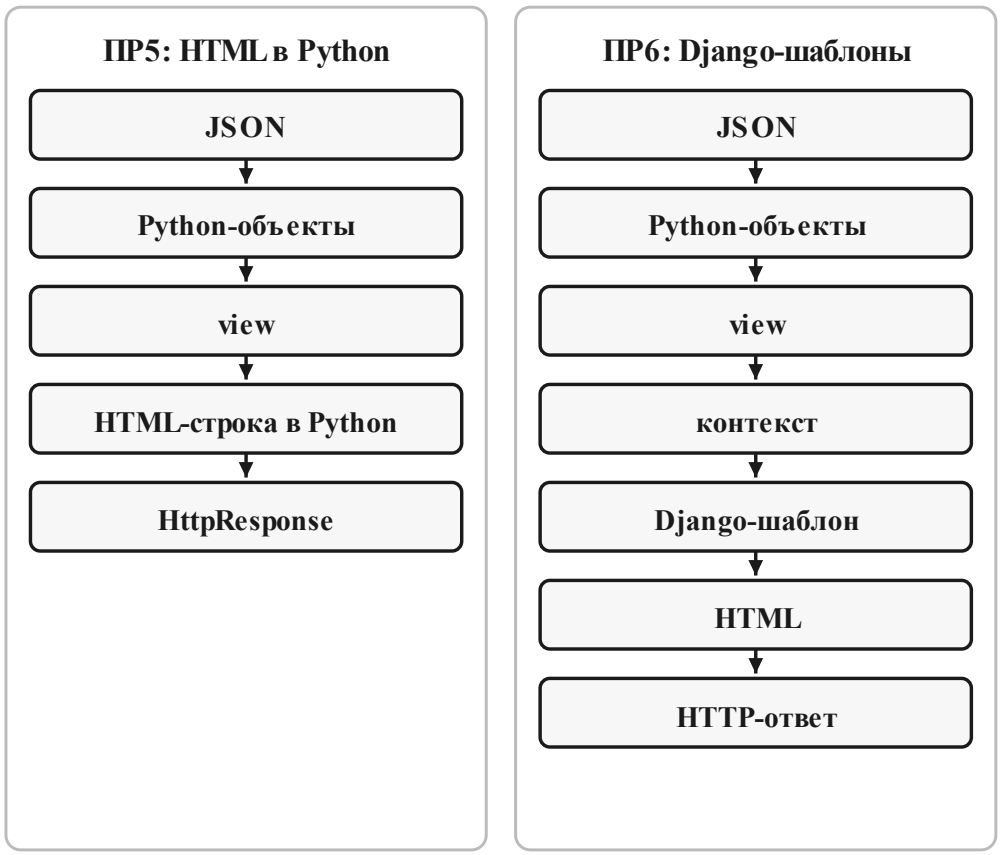


Рисунок 2 – Архитектура веб-части проекта до и после ПР6

На этом этапе Django ORM ещё не используется. Источник данных остаётся прежним – JSON-файлы и объекты `Room`, `User` и `Booking`, созданные в ПР3.

Примечание. Переход к моделям Django и базе данных выполняется в следующих практических работах.

3.14. Что должно измениться в проекте

К концу ПР6 необходимо устранить зависимость веб-части проекта от функции `page()` из ПР5. То есть конструкции вида:

```
return HttpResponse(page(...))
```

в веб-представлениях `Roomly` использоваться не должны.

Вместо них должны применяться:

```
return render(request, "имя_шаблона.html", context)
```

При этом:

- HTML переносится из Python в файлы .html;
- повторяющаяся структура переносится в `base.html`;
- повторяющиеся фрагменты оформляются через `include`;

- данные передаются через `context`;
- циклы и условия реализуются средствами Django Template Language;
- ссылки формируются через `{% url %}`;
- форматирование данных выполняется фильтрами;
- CSS, изображения и JavaScript подключаются как static-файлы.

Таким образом, ПР6 является не просто добавлением нескольких HTML-файлов, а перестройкой слоя представления Django-приложения.

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

В рамках ПР6 выполняются материалы уроков «HTML и шаблоны Django» и «Финальный проект спринта Блогикум» Раздела 04 «Основы веб-разработки» курса Яндекс Практикума «Бэкенд-разработка». Перечень материалов приведен в таблице: материалы отмечаются по мере выполнения. Полный чек-лист для контроля – в подразделе 7.1.

№	Материал Яндекс Практикума	Выполнить
1	HTML и шаблоны Django / HTML в Django: шаблоны веб-страниц	✓
2	HTML и шаблоны Django / HTML-шаблоны из частей: теги include, extends, block	✓
3	HTML и шаблоны Django / Словарь контекста, ветвление и циклы в шаблонах	✓
4	HTML и шаблоны Django / Ссылки в шаблонах: name и namespace	✓
5	HTML и шаблоны Django / Фильтры в шаблонах: модифицируем контекст страницы	✓
6	HTML и шаблоны Django / Подключение стилей, картинок и скриптов к шаблонам	✓
7	HTML и шаблоны Django / Из тренажёра на компьютер. Настройка VS Code	✓
8	HTML и шаблоны Django / Правила оформления HTML и шаблонов Django	✓
9	HTML и шаблоны Django / Создание шаблонов из HTML-документов. Практика	✓
10	Финальный проект спринта Блогикум / Что предстоит сделать	✓
11	Финальный проект спринта Блогикум / Проект. Приложение «Блогикум». Часть 1	✓
12	Финальный проект спринта Блогикум / Обратная связь	✓

Материалы Яндекс Практикума выполняются непосредственно в курсе «Бэкенд-разработка». Практическая часть индивидуального проекта выполняется параллельно и должна закреплять соответствующие технологии на собственном проекте.

5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

5.1. Общая постановка задания

В ПР6 необходимо переработать веб-интерфейс индивидуального проекта, созданный в ПР5, и перевести его с формирования HTML-кода в Python на использование системы шаблонов Django.

В ПР5 веб-часть проекта формировала HTML-строки непосредственно в Python-коде. В ПР6 необходимо реализовать шаблонную архитектуру: `view`-функция подготавливает контекст и вызывает Django-шаблон, а HTML формирует шаблон (рис. 2).

В качестве результата необходимо получить полноценный шаблонный интерфейс существующего проекта с общей структурой страниц, навигацией, динамическим отображением данных, условиями, циклами, фильтрами и собственными статическими файлами.

Примечание. Функция `page()` из ПР5 в веб-части проекта после выполнения ПР6 использоваться не должна.

5.2. Создание структуры шаблонов

Для каждого Django-приложения необходимо создать каталог `templates`. Для приложения `homepage`:

```
homepage/
├── templates/
│   └── homepage/
│       └── index.html
│
├── static/
│   └── homepage/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   └── main.js
│       └── img/
│           └── logo.png
│
├── views.py
├── urls.py
└── ...
```

Для приложения `rooms`:

```
rooms/
├── templates/
│   └── rooms/
│       ├── room_list.html
│       ├── room_detail.html
│       └── includes/
│           └── room_card.html
│
├── views.py
├── urls.py
└── ...
```

Для приложения `bookings`:

```
bookings/
├── templates/
│   └── bookings/
│       ├── booking_list.html
│       ├── booking_detail.html
│       └── includes/
│           └── booking_status.html
│
├── views.py
├── urls.py
└── ...
```

Кроме шаблонов отдельных приложений, в проекте необходим каталог шаблонов уровня проекта. В нём размещаются шаблоны, общие для всего сайта: базовый шаблон `base.html` и подключаемые фрагменты, используемые на всех страницах (например, навигация). Каталог создаётся в корне проекта рядом с `manage.py`:

```
room_booking/
├── templates/
│   ├── includes/
│   │   └── navigation.html
│   └── base.html
├── homepage/
├── rooms/
├── bookings/
└── manage.py
```

Чтобы Django находил шаблоны в каталоге уровня проекта, в `settings.py` необходимо указать этот каталог в параметре `DIRS` списка `TEMPLATES`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
```

```
        "APP_DIRS": True,
        # ... остальные параметры остаются по умолчанию
    },
]
```

Параметр `APP_DIRS` отвечает за поиск шаблонов в каталогах приложений, а параметр `DIRS` добавляет к поиску каталоги уровня проекта. Остальные настройки `TEMPLATES`, созданные Django по умолчанию, изменять не требуется.

Такая организация позволяет не смешивать шаблоны разных приложений и уменьшает вероятность совпадения имён файлов.

5.3. Создание базового шаблона

Файл `templates/base.html` должен содержать общую HTML-структуру сайта. Базовый шаблон является общим для всех страниц проекта, поэтому он размещается в каталоге шаблонов уровня проекта (раздел 5.2), а не внутри отдельного приложения. На этом этапе не следует усложнять: сначала необходимо перенести в базовый шаблон структуру, которую в ПР5 формировала функция `page()`:

- объявление HTML-документа;
- кодировку;
- название страницы;
- подключение Bootstrap;
- подключение собственного CSS;
- навигационное меню;
- основной контейнер страницы;
- подключение JavaScript;
- подвал страницы.

Базовый шаблон заполняется в два приёма. Сначала создаётся минимальный каркас: HTML-структура документа, подключение Bootstrap, навигация и контейнер с блоками. Базовый шаблон должен содержать блоки, которые дочерние страницы смогут переопределять:

```
{% block title %}Roomly{% endblock %}
{% block content %}
{% endblock %}
```

Минимальная структура `base.html`:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta    name="viewport"    content="width=device-width,    initial-
scale=1">
```

```
    <title>{% block title %}Roomly{% endblock %}</title>
    <link
        rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3
              /dist/css/bootstrap.min.css"
    >
</head>
<body>
    <nav class="nav">
        <a class="nav-link" href="/">Главная</a>
        <a class="nav-link" href="/rooms/">Помещения</a>
        <a class="nav-link" href="/bookings/">Бронирования</a>
    </nav>

    <main class="container">
        {% block content %}
        {% endblock %}
    </main>

</body>
</html>
```

Примечание. Навигация здесь пока записана обычными адресами и находится прямо в `base.html` – это осознанное упрощение. Сначала важно увидеть сам механизм наследования: `base.html` содержит `{% block title %}` и `{% block content %}`, а дочерние страницы их переопределяют. Навигация будет вынесена во включаемый шаблон `includes/navigation.html` в разделе 5.9.

Название проекта и элементы навигации не должны дублироваться в каждом дочернем шаблоне.

5.4. Создание главной страницы

Главная страница должна содержать:

- название проекта;
- краткое описание системы;
- назначение Roomly;
- навигацию к основным разделам;
- ссылки на помещения и бронирования.

HTML-код общей структуры страницы в `index.html` дублировать не следует. Файл `homepage/templates/homepage/index.html` должен наследовать базовый шаблон:

```
{% extends "base.html" %}

{% block title %}
    Главная – Roomly
```

```
{% endblock %}

{% block content %}
    <h1 class="display-4">Roomly</h1>

    <p class="lead">
        Система бронирования помещений.
    </p>
    <p>Основные разделы:</p>
    <a href="/rooms/" class="btn btn-primary me-2">
        Помещения
    </a>
    <a href="/bookings/" class="btn btn-secondary">
        Бронирования
    </a>
{% endblock %}
```

Здесь `index.html` подключает `base.html` через `{% extends %}` и переопределяет его блоки. Файл `index.html` не содержит ни объявления документа, ни `<head>`, ни навигации – общая структура приходит из базового шаблона.

5.5. Перевод homepage на render() и судьба функции page()

Шаблоны главной страницы готовы, но представление по-прежнему формирует HTML в Python. Теперь следует переделать `homepage/views.py`.

Сейчас в `homepage/views.py`:

```
def index(request):
    content = """
    ...
    """
    return HttpResponse(page("Roomly", content))
```

Это больше не нужно. Функция `index()` должна стать:

```
from django.shortcuts import render


def index(request):
    return render(
        request,
        "homepage/index.html",
    )
```

На этом изменения `homepage/views.py` завершены.

Примечание. Контекст здесь пока не нужен: главная страница не выводит динамических данных. Вызов `render()` без контекста просто формирует страницу из шаблона.

Функцию `page()` окончательно удалять пока не нужно. После изменения `index()` она перестанет использоваться в `homepage/views.py`, но её по-прежнему импортируют представления `rooms/views.py` и `bookings/views.py`:

```
from homepage.views import page
```

Если удалить `page()` сейчас, остальные страницы проекта сломаются. Последовательность перехода выглядит так:

```
ПР5:  page() -> homepage
ПР6:  base.html -> homepage -> rooms -> bookings -> удаляем page()
```

Функция `page()` удаляется в самом конце работы, когда весь проект перейдёт на `render()` (раздел 5.13).

Запустите сервер и откройте главную страницу:

```
python manage.py runserver

http://127.0.0.1:8000/
```

Главная должна выглядеть практически так же, как в ПР5, но архитектурно произошло важное изменение:

```
Было:  index() -> content = """HTML""" -> page() -> HttpResponse()
Стало: index() -> render() -> homepage/index.html -> base.html
```

Перед переходом к следующему разделу проверьте три вещи:

- главная страница открывается по адресу /;
- заголовок страницы в браузере – «Главная – Roomly»;
- ссылки «Помещения» и «Бронирования» на странице по-прежнему работают.

5.6. Перенос страниц помещений на шаблоны

Страницы приложений `rooms` и `bookings` переносятся на шаблоны по единой схеме: `view`-функция подготавливает контекст и вызывает `render()`, шаблон формирует HTML-страницу. Начнём с приложения `rooms`: сначала перенесём список помещений – заодно уберём HTML из `views.py` и передадим данные через контекст.

5.6.1. Список помещений и функция render()

Сейчас в `rooms/views.py` список помещений формируется непосредственно в Python-коде:

```
def rooms(request):
    items = ""
    for room in load_rooms("data/rooms.json"):
        items += f'<li class="list-group-item">{room.name} – вместимость
{room.capacity} мест</li>'

    content = f"""
```

```
    <h1>Помещения</h1>
    <ul class="list-group">{items}</ul>
    """
    return HttpResponse(page("Roomly – помещения", content))
```

Задача – разделить эту работу на две части:

```
views.py -> получить rooms -> передать rooms в context -> render()
room_list.html -> {% for room in rooms %}
```

Создадим каталог `rooms/templates/rooms/` и файл `room_list.html`:

```
{% extends "base.html" %}

{% block title %}
    Помещения – Roomly
{% endblock %}

{% block content %}
    <h1>Помещения</h1>

    <ul class="list-group">
        {% for room in rooms %}
            <li class="list-group-item">
                {{ room.name }} – вместимость {{ room.capacity }} мест
            </li>
        {% empty %}
            <li class="list-group-item">
                Помещения не найдены.
            </li>
        {% endfor %}
    </ul>
{% endblock %}
```

В шаблоне появились две новые конструкции Django Template Language.

- `{{ room.name }}` – вывод значения: Django сам обращается к атрибуту объекта `Room`, в `{{ room.capacity }}` будет выведено значение `room.capacity`;
- `{% for room in rooms %}` – цикл по коллекции: Python-цикла во `view` больше нет.

Теперь переделаем функцию `rooms()` в `rooms/views.py`. Она должна:

- получить список помещений через существующую функцию загрузки данных;
- подготовить контекст;
- передать контекст в шаблон через `render()`.

Например:

```
from django.shortcuts import render

from storage import load_rooms


def rooms(request):
    rooms_list = load_rooms("data/rooms.json")

    context = {
        "rooms": rooms_list,
    }

    return render(
        request,
        "rooms/room_list.html",
        context,
    )
```

В результате HTML-разметка списка помещений должна находиться не в `views.py`, а в:

```
rooms/templates/rooms/room_list.html
```

Посмотрим на разницу. В ПР5 Python одновременно перебирал помещения, создавал HTML и формировал страницу. Теперь Python занимается только данными:

```
rooms_list = load_rooms("data/rooms.json")
context = {
    "rooms": rooms_list,
}
```

А HTML находится в файле `room_list.html`, и именно шаблон выполняет `{% for room in rooms %}`.

Примечание. В `rooms/views.py` остались импорты `HttpResponse`, `page` и другие, которые списку помещений уже не нужны. Пока их не удаляем: `room_detail()` ещё использует их, а аккуратная очистка импортов будет выполнена после переноса страницы помещения (раздел 5.6.3).

Запустим сервер и откроем `http://127.0.0.1:8000/rooms/`. Список должен выглядеть так же, как в ПР5: названия помещений и вместимость. Проверим и обработку пустого списка: если временно передать в шаблон пустую коллекцию,

```
return render(
    request,
    "rooms/room_list.html",
    {"rooms": []},
)
```

то должно появиться сообщение «Помещения не найдены.». Специально менять код ради этой проверки не обязательно: конструкция `{% empty %}` уже работает.

5.6.2. Карточка помещения и {% include %}

Повторяющийся HTML-код одного помещения необходимо вынести в:

```
rooms/templates/rooms/includes/room_card.html
```

Сейчас `room_list.html` сам отвечает и за перебор помещений, и за HTML одного помещения. Разделим ответственность: `room_list.html` перебирает коллекцию и подключает включаемый шаблон, а `room_card.html` отображает один объект `Room`. Создадим каталог `includes` и файл `room_card.html`:

```
rooms/
└── templates/
    └── rooms/
        ├── room_list.html
        └── includes/
            └── room_card.html
```

В карточке необходимо вывести данные объекта `Room`. Запишем в `room_card.html`:

```
<article class="card mb-3">
    <div class="card-body">
        <h2 class="card-title">
            {{ room.name }}
        </h2>

        <p class="card-text">
            Вместимость: {{ room.capacity }} мест
        </p>

        <a
            href="{% url 'rooms:detail' room.id %}"
            class="btn btn-primary"
        >
            Подробнее
        </a>
    </div>
</article>
```

Обратим внимание на новую конструкцию:

```
{% url 'rooms:detail' room.id %}
```

Вместо того чтобы писать адрес вида `/rooms/1/`, у Django просят построить URL для маршрута с именем `rooms:detail`, передав ему `room.id`. Это правильнее: URL формируется на основании `urls.py`, а не жёстко зашит в HTML.

Примечание. Изменения в URL-файлах при переносе на шаблоны не требуются: имена маршрутов и пространства имён (`app_name`) уже определены в ПР5. Необходимо убедиться, что имена маршрутов соответствуют тегам `{% url %}`.

В файле `rooms/urls.py` должны быть определены пространство имён и маршруты:

```
from django.urls import path

from . import views


app_name = "rooms"

urlpatterns = [
    path("", views.rooms, name="list"),
    path("<int:room_id>/", views.room_detail, name="detail"),
]
```

Именно из `app_name = "rooms"` и `name="detail"` складывается имя `rooms:detail`, которое используется в карточке. Аналогично проверьте `homepage/urls.py`:

```
from django.urls import path
from . import views


app_name = "homepage"

urlpatterns = [
    path("", views.index, name="index"),
]
```

Основной шаблон списка помещений должен подключать карточку:

```
{% for room in rooms %}
    {% include "rooms/includes/room_card.html" %}
{% empty %}
    <p>Помещения не найдены.</p>
{% endfor %}
```

Таким образом, список помещений отвечает за перебор коллекции, а отдельный шаблон – за отображение одного элемента.

Примечание. Откуда `room_card.html` знает, что такое `room`? Из внешнего цикла: Django при каждой итерации передаёт текущий объект `room` в подключаемый шаблон. Поэтому на каждой итерации включаемый шаблон получает очередной объект: `Room(id=1)`, затем `Room(id=2)` и так далее.

Откроем `http://127.0.0.1:8000/rooms/`. У каждого помещения должна появиться кнопка «Подробнее», ведущая на `/rooms/1/` или другой существующий

идентификатор. Пока на ней может открываться старая HTML-версия `room_detail()`. Для каждого помещения карточка выводит минимум:

- название;
- вместимость;
- ссылку на страницу помещения.

5.6.3. Страница отдельного помещения и обработка 404

Страница `/rooms/<id>/` должна быть полностью переведена на шаблоны. Сейчас `room_detail()` всё ещё формирует HTML в Python. Во `view` нужно оставить только четыре действия:

- загрузку помещений;
- поиск помещения;
- подготовку контекста;
- вызов `render()`.

А HTML перенести в шаблон `room_detail.html`. Шаблон `rooms/templates/rooms/room_detail.html` должен наследовать `base.html`. На странице необходимо отобразить:

- название помещения;
- вместимость;
- описание, если оно предусмотрено моделью проекта;
- информацию о доступности;
- ссылку обратно к списку помещений.

Запишите в `room_detail.html`:

```
{% extends "base.html" %}

{% block title %}
    {% if room %}
        {{ room.name }} – Roomly
    {% else %}
        Помещение не найдено – Roomly
    {% endif %}
{% endblock %}

{% block content %}
    {% if room %}
        <div class="card">
            <div class="card-body">
                <h1 class="card-title">
                    {{ room.name }}
                </h1>
```

```
                <p class="card-text">
                    <strong>ID:</strong> {{ room.id }}
                </p>

                <p class="card-text">
                    <strong>Вместимость:</strong>
                    {{ room.capacity }} мест
                </p>
            </div>
        </div>
    {% else %}
        <h1 class="text-danger">
            Помещение не найдено
        </h1>
        <p>
            Помещение с указанным идентификатором отсутствует.
        </p>
    {% endif %}

    <a
        href="{% url 'rooms:list' %}"
        class="btn btn-outline-secondary mt-3"
    >
        ← к списку помещений
    </a>
{% endblock %}
```

Почему здесь `{% if room %}`? Потому что по условию ПР6 ситуация отсутствия объекта должна обрабатываться именно в шаблоне. Во `view` определяется `room = find_room_by_id(...)`; если помещение не найдено, `room is None`, в шаблон передаётся `{"room": None}` и срабатывает ветка `{% else %}`.

Теперь заменим старую функцию `room_detail()` во `view`. View-функция получает объект помещения:

```
def room_detail(request, room_id):
    rooms_list = load_rooms("data/rooms.json")
    room = find_room_by_id(rooms_list, room_id)

    context = {
        "room": room,
    }

    return render(
        request,
```

```
        "rooms/room_detail.html",
        context,
        status=404 if room is None else 200,
    )
```

Функция `find_room_by_id()` реализована в ПР3.

Примечание. Сохраним поведение ПР5: если помещение не найдено, ответ возвращается с кодом 404. Проверка `room is None` выполняется во `view`, сообщение пользователю выводится в шаблоне через `{% if %} / {% else %}`, а в вызов `render()` при этом передается параметр `status=404`.

Теперь начало `rooms/views.py` должно прийти к следующему виду:

```
from django.shortcuts import render

from models.rooms import find_room_by_id
from storage import load_rooms
```

Удалите ставшие ненужными импорты:

```
from datetime import date
from django.http import HttpResponse
from homepage.views import page
from models.bookings import is_room_available
from storage import load_bookings
```

В результате весь `rooms/views.py` должен выглядеть примерно так:

```
from django.shortcuts import render

from models.rooms import find_room_by_id
from storage import load_rooms


def rooms(request):
    rooms_list = load_rooms("data/rooms.json")

    context = {
        "rooms": rooms_list,
    }

    return render(
        request,
        "rooms/room_list.html",
        context,
    )


def room_detail(request, room_id):
    rooms_list = load_rooms("data/rooms.json")
```

```
    room = find_room_by_id(
        rooms_list,
        room_id,
    )

    context = {
        "room": room,
    }

    return render(
        request,
        "rooms/room_detail.html",
        context,
        status=404 if room is None else 200,
    )
```

Примечание. HTML-кода **в** `rooms/views.py` больше нет вообще. Лишние импорты, которые сознательно оставлялись после переноса списка помещений, теперь удалены: они были нужны только старой `room_detail()`.

Откроем существующее помещение, например `http://127.0.0.1:8000/rooms/1/`. Должна появиться карточка помещения, а заголовок вкладки браузера – «Комната 101 – Roomly» (по фактическому названию помещения).

Теперь введем идентификатор, которого точно нет: `http://127.0.0.1:8000/rooms/9999/`. Должна открыться страница «Помещение не найдено». Здесь два независимых результата проверки:

- внешний результат: в браузере отображается собственная страница ошибки;
- HTTP-результат: ответ сервера должен иметь статус 404.

Именно это обеспечивает параметр `status` в вызове `render()`:

```
status=404 if room is None else 200
```

Примечание. Просто вывести текст «Помещение не найдено» недостаточно – HTTP-ответ всё равно должен иметь статус 404.

На этом этапе требования по приложения `rooms` практически полностью выполнены.

5.7. Условное отображение данных

В шаблонах `Roomly` необходимо использовать конструкции `{% if %}`. Например, для бронирования:

```
{% if booking.is_cancelled %}
    <span>Отменено</span>
{% else %}
    <span>Активно</span>
{% endif %}
```

Для списка помещений необходимо предусмотреть ситуацию отсутствия данных:

```
{% if rooms %}
    ...
{% else %}
    <p>Помещения не найдены.</p>
{% endif %}
```

При использовании `{% for ... empty %}` отдельный `{% if %}` для этой же проверки не требуется.

5.8. Перенос страниц бронирований на шаблоны

Страницы приложения `bookings` переносятся по той же схеме, что и помещения. Здесь дополнительно отрабатываются условия `{% if %}`, включаемый шаблон статуса и динамические данные предметной области: `booking.room.name`, `booking.booking_date`, `booking.user.name`, `booking.is_cancelled`.

5.8.1. Список бронирований

Переведем на шаблоны страницу `/bookings/` по той же схеме, что и список помещений: сначала шаблон с циклом, затем `view`-функция.

View-функция должна передавать список бронирований:

```
def bookings(request):
    rooms_list = load_rooms("data/rooms.json")
    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
        "data/bookings.json",
        rooms_list,
        users_list,
    )

    context = {
        "bookings": bookings_list,
    }

    return render(
        request,
        "bookings/booking_list.html",
        context,
    )
```

В шаблоне необходимо использовать цикл:

```
{% for booking in bookings %}
    ...
{% empty %}
```

```
    <p>Бронирования не найдены.</p>
{% endfor %}
```

Для каждого бронирования необходимо отобразить существующие данные предметной области:

- идентификатор;
- помещение;
- пользователя;
- дату;
- статус;
- ссылку на подробную информацию.

Конкретные имена атрибутов должны соответствовать классам `Booking`, `Room` и `User`, созданным в ПР3 (например, `booking.room.name`, `booking.user.name`, `booking.is_cancelled`).

5.8.2. Статус бронирования и {% include %}

Для отображения статуса необходимо создать:

```
bookings/templates/bookings/includes/booking_status.html
```

В шаблоне необходимо предусмотреть как минимум два состояния:

```
{% if booking.is_cancelled %}
    <span class="booking-status booking-status-cancelled">
        Отменено
    </span>
{% else %}
    <span class="booking-status booking-status-active">
        Активно
    </span>
{% endif %}
```

Основной шаблон бронирования должен подключать этот фрагмент через:

```
{% include "bookings/includes/booking_status.html" %}
```

При этом определение статуса должно использовать существующее состояние объекта `Booking`.

Один и тот же фрагмент отображения статуса используется и в списке бронирований, и на странице конкретного бронирования.

5.8.3. Маршруты бронирований: проверка вместо изменений

В рамках переноса интерфейса на шаблоны структура маршрутов приложения `bookings` сохраняется – дополнительных изменений в `bookings/urls.py` не требуется. Необходимо убедиться, что в файле определено пространство имён приложения и имена маршрутов:

```
app_name = "bookings"

urlpatterns = [
    path("<int:booking_id>/", views.booking_detail, name="detail"),
]
```

Это позволяет формировать ссылки на страницы отдельных бронирований непосредственно в шаблоне с помощью тега `{% url %}`:

```
<a href="{% url 'bookings:detail' booking.id %}">
    Бронирование №{{ booking.id }}
</a>
```

Параметр `booking.id` передаётся в URL как значение параметра `booking_id`, заданного в маршруте. Маршрут списка бронирований в проекте имеет имя `bookings`:

```
path("", views.bookings, name="bookings"),
```

Поэтому ссылки на список бронирований в шаблонах формируются так:

```
{% url 'bookings:bookings' %}
```

Примечание. Проверьте фактические имена в своих `urls.py` и используйте именно их в тегах `{% url %}`.

5.8.4. Страница отдельного бронирования

Переведем страницу `/bookings/<id>/` на шаблон. Создадим файл `bookings/templates/bookings/booking_detail.html`:

```
{% extends "base.html" %}

{% block title %}
    {% if booking %}
        Бронирование №{{ booking.id }} – Roomly
    {% else %}
        Бронирование не найдено – Roomly
    {% endif %}
{% endblock %}

{% block content %}
    {% if booking %}
        <div class="card">
            <div class="card-body">
                <h1 class="card-title">
                    Бронирование №{{ booking.id }}
                </h1>

                <p class="card-text">
                    <strong>Помещение:</strong>
```

```
                    {{ booking.room.name }}
                </p>

                <p class="card-text">
                    <strong>Дата:</strong>
                    {{ booking.booking_date }}
                </p>

                <p class="card-text">
                    <strong>Пользователь:</strong>
                    {{ booking.user.name }}
                </p>

                <p class="card-text">
                    <strong>Статус:</strong>
                    {% include "bookings/includes/booking_status.html"
%}
                </p>
            </div>
        </div>
    {% else %}
        <h1 class="text-danger">
            Бронирование не найдено
        </h1>

        <p>
            Бронирование с указанным идентификатором отсутствует.
        </p>
    {% endif %}

    <a
        href="{% url 'bookings:bookings' %}"
        class="btn btn-outline-secondary mt-3"
    >
        ← к списку бронирований
    </a>
{% endblock %}
```

Здесь специально используется уже созданный включаемый шаблон `booking_status.html`: один фрагмент отображает статус и в списке, и на странице бронирования. Ссылка «← к списку бронирований» формируется через `{% url 'bookings:bookings' %}`. Теперь заменим старую функцию `booking_detail()` во `view`. Новая версия функции:

```
from django.shortcuts import render
from models.bookings import find_booking_by_id
```

```
from storage import load_bookings, load_rooms, load_users
def booking_detail(request, booking_id):
    rooms_list = load_rooms("data/rooms.json")
    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
        "data/bookings.json",
        rooms_list,
        users_list,
    )
    booking = find_booking_by_id(
        bookings_list,
        booking_id,
    )
    context = {
        "booking": booking,
    }
    return render(
        request,
        "bookings/booking_detail.html",
        context,
        status=404 if booking is None else 200,
    )
```

Примечание. Раньше в `booking_detail()` использовался искусственно созданный пользователь `User(user_id=1, name="User", email="user@example.com")`. Теперь используем реальную загрузку `users_list = load_users("data/users.json")`, как и в функции `bookings()`.

5.8.5. Окончательный вид bookings/views.py и проверка

После замены обеих функций файл `bookings/views.py` приводится к окончательному виду:

```
from django.shortcuts import render

from models.bookings import find_booking_by_id
from storage import load_bookings, load_rooms, load_users


def bookings(request):
    rooms_list = load_rooms("data/rooms.json")
    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
        "data/bookings.json",
        rooms_list,
        users_list,
    )
```

```
    context = {
        "bookings": bookings_list,
    }

    return render(
        request,
        "bookings/booking_list.html",
        context,
    )


def booking_detail(request, booking_id):
    rooms_list = load_rooms("data/rooms.json")
    users_list = load_users("data/users.json")
    bookings_list = load_bookings(
        "data/bookings.json",
        rooms_list,
        users_list,
    )

    booking = find_booking_by_id(
        bookings_list,
        booking_id,
    )

    context = {
        "booking": booking,
    }

    return render(
        request,
        "bookings/booking_detail.html",
        context,
        status=404 if booking is None else 200,
    )
```

Из `bookings/views.py` исчезают импорты:

```
from django.http import HttpResponse
from homepage.views import page
from models import User
```

Проверим результат:

- список бронирований открывается по адресу `/bookings/`;
- страница существующего бронирования (`/bookings/1/`) показывает номер, помещение, дату, пользователя, статус и ссылку «← к списку бронирований»;

- страница несуществующего бронирования (/bookings/9999/) показывает сообщение «Бронирование не найдено», а HTTP-ответ имеет статус 404.

На этом этапе полностью переведены на шаблоны: главная страница, список помещений, карточка помещения, страница помещения, список бронирований, статус бронирования, страница бронирования. HTML этих страниц больше не формируется во views.py.

5.9. Общая навигация сайта

Теперь вынесем навигацию из base.html в отдельный подключаемый шаблон. Навигация одинакова на всех страницах проекта – идеальный кандидат на {% include %}. Создадим файл:

```
templates/includes/navigation.html
```

Запишем в navigation.html:

```
<nav class="nav">
    <a class="nav-link" href="{% url 'homepage:index' %}">Главная</a>
    <a class="nav-link" href="{% url 'rooms:list' %}">Помещения</a>
    <a class="nav-link" href="{% url 'bookings:bookings' %}">Бронирования</a>
</nav>
```

Ссылка на главную также формируется через именованный маршрут homepage:index, а не обычным адресом «/» – жёстко заданные адреса в навигации использоваться не должны. Подключим navigation.html в base.html. Сейчас в base.html навигация прописана непосредственно:

```
<nav class="nav">
    <a class="nav-link" href="/">Главная</a>
    <a class="nav-link" href="/rooms/">Помещения</a>
    <a class="nav-link" href="/bookings/">Бронирования</a>
</nav>
```

Этот блок удалим и заменим на:

```
{% include "includes/navigation.html" %}
```

В результате base.html должен выглядеть так:

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Roomly{% endblock %}</title>
    <link
```

```
        rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3
              /dist/css/bootstrap.min.css"
    >
</head>
<body>
    {% include "includes/navigation.html" %}

    <main class="container">
        {% block content %}
        {% endblock %}
    </main>

</body>
</html>
```

Проверим результат. Перезапуск сервера обычно не требуется – достаточно обновить страницу. Навигация должна отображаться на главной странице, `/rooms/` и `/bookings/`. Особенно проверим, что ссылки «Помещения» и «Бронирования» работают: теперь они формируются средствами Django, а не вручную.

Теперь `base.html` отвечает только за общую структуру страницы, а `navigation.html` – за навигацию. Структура шаблонов принимает вид (рис. 3):

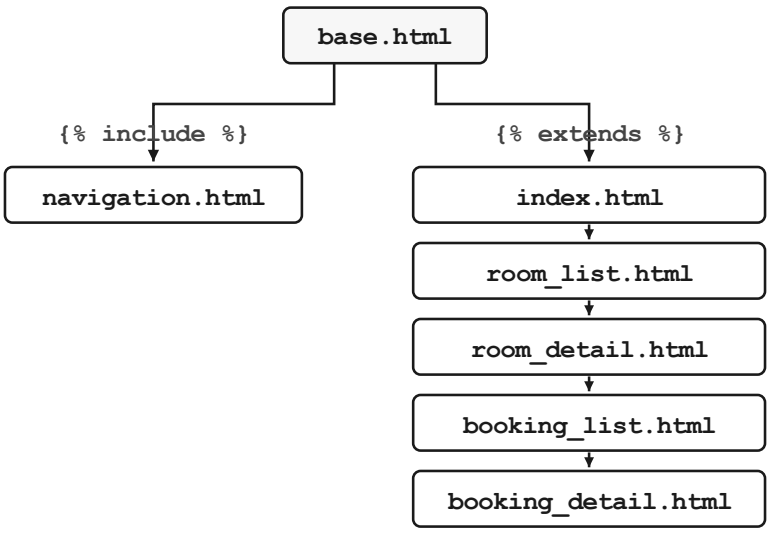


Рисунок 3 – Связь базового шаблона с дочерними шаблонами

5.10. Фильтры шаблонов

Фильтры необходимо использовать на реальных данных проекта. Минимально в проекте должны быть продемонстрированы следующие случаи.

- Количество элементов.

Откроем `rooms/templates/rooms/room_list.html`. Сейчас после заголовка сразу идёт цикл. Добавьте между ними:

```
<p>
    Всего помещений: {{ rooms|length }}
</p>
```

Проверьте `/rooms/` – должна появиться строка «Всего помещений: 5» (число зависит от данных проекта). Здесь используется фильтр `|length` – он получает количество элементов списка `rooms`.

- Значение по умолчанию

Откроем `rooms/templates/rooms/includes/room_card.html`. Заменим `{{ room.name }}` на:

```
{{ room.name|default:"Без названия" }}
```

Это демонстрация фильтра `default`. Если `room.name` содержит значение – оно выводится как обычно; если значение отсутствует или пустое – выводится «Без названия». Модель `Room` и проверки в Python не меняются: это именно форматирование представления данных, а не бизнес-логика.

Фильтры не должны использоваться для реализации операций, которые относятся к бизнес-логике приложения.

5.11. Статические файлы проекта

5.11.1. Структура и подключение статических файлов

В приложении `homepage` создадим структуру каталогов:

```
homepage/
└── static/
    └── homepage/
        ├── css/
        │   └── style.css
        ├── js/
        │   └── main.js
        └── img/
            └── logo.png
```

Файл `logo.png` может быть заменён другим изображением, относящимся к предметной области проекта.

Подключим статику в `base.html`. Самой первой строкой файла добавим тег:

```
{% load static %}
```

Без этого тега конструкции `{% static %}` работать не будут.

В `<head>` после подключения Bootstrap добавим собственный CSS:

```
<link
    rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3
```

```
          /dist/css/bootstrap.min.css"
>
<link
    rel="stylesheet"
    href="{% static 'homepage/css/style.css' %}"
>
```

Логотип можно разместить в `base.html` перед навигацией:

```
<body>
    <img
        src="{% static 'homepage/img/logo.png' %}"
        alt="Roomly"
    >

    {% include "includes/navigation.html" %}
```

Пока это просто проверка подключения изображения: позже собственный CSS аккуратно оформит логотип как часть шапки. Собственный CSS должен использоваться для оформления элементов интерфейса. Bootstrap можно сохранить для базовой вёрстки, однако результат текущей практической работы должен содержать также собственные стили проекта.

5.11.2. Собственный CSS проекта

Файл `homepage/static/homepage/css/style.css` должен содержать стили, относящиеся непосредственно к проекту. Например:

```
body {
    min-height: 100vh;
}

.room-card {
    margin-bottom: 1rem;
    padding: 1rem;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.booking-status {
    font-weight: 600;
}

.roomly-logo {
    width: 120px;
    height: auto;
    margin: 1rem 0;
}
```

Логотипу в base.html добавим класс roomly-logo:

```
<img
    src="{% static 'homepage/img/logo.png' %}"
    alt="Roomly"
    class="roomly-logo"
>
```

Классы .room-card и .booking-status пока не используются шаблонами – они будут связаны с карточками помещений и статусами бронирований в разделе 5.11.4. Стили необходимо адаптировать к собственному интерфейсу.

Примечание. Перед подключением JavaScript необходимо проверить: фильтры length и default добавлены; структура static создана; CSS **и** logo.png подключены; логотип появляется на страницах, а собственный CSS работает.

5.11.3. Подключение JavaScript

Файл homepage/static/homepage/js/main.js должен быть подключён к базовому шаблону. JavaScript должен выполнять хотя бы одну небольшую функцию интерфейса. Например, можно реализовать вывод текущего года в элемент:

```
<span id="current-year"></span>
```

Создадим файл homepage/static/homepage/js/main.js со следующим кодом:

```
const yearElement = document.getElementById("current-year");

if (yearElement) {
    yearElement.textContent = new Date().getFullYear();
}
```

JavaScript автоматически определяет текущий год и выводит его в нужный элемент страницы. Добавим подвал в base.html – после закрывающего </main>:

```
<footer class="container mt-4 mb-3">
    <p>
        © <span id="current-year"></span> Roomly
    </p>
</footer>
```

Подключим JavaScript перед закрывающим </body>:

```
<script src="{% static 'homepage/js/main.js' %}"></script>
```

Тег {% static %} работает потому, что в самом начале base.html уже есть {% load static %}.

Откроем главную страницу: внизу должно появиться, например, «© 2026 Roomly». Год не прописан в HTML – его устанавливает main.js. Затем проверим

/rooms/ и /bookings/: обе страницы наследуют base.html, поэтому JavaScript работает и там.

Допускается реализовать другое небольшое клиентское поведение, если оно относится к интерфейсу проекта. Цель задания – продемонстрировать правильное подключение собственного JavaScript-файла через механизм Django static files.

5.11.4. Оформление карточек и статуса собственными стилями

Теперь собственный CSS доводится до реального использования в интерфейсе. Откроем rooms/templates/rooms/includes/room_card.html. Сейчас корневой элемент:

```
<article class="card mb-3">
```

Заменим на:

```
<article class="card room-card mb-3">
```

Откроем bookings/templates/bookings/includes/booking_status.html и добавим к элементам статуса классы Bootstrap badge и собственные классы проекта:

```
{% if booking.is_cancelled %}

    <span class="badge bg-secondary booking-status booking-status-cancelled">
        Отменено
    </span>

{% else %}

    <span class="badge bg-success booking-status booking-status-active">
        Активно
    </span>

{% endif %}
```

Теперь эти элементы одновременно используют Bootstrap для базового оформления и собственный CSS Roomly для оформления проекта.

Проверим /rooms/ и /bookings/: у помещений должны появиться собственные стили карточек, у статусов – собственное начертание. Итог: все три типа статических ресурсов реально используются в проекте.

Ресурс	Где используется
style.css	base.html, room_card.html, booking_status.html
main.js	base.html
logo.png	base.html

5.12. Обновление README.md

В `README.md` необходимо отразить состояние проекта после ПР6. В описание текущей реализации следует добавить:

- использование Django Template Language;
- базовый шаблон `base.html`;
- наследование шаблонов;
- блоки block;
- включаемые шаблоны include;
- передачу данных через контекст;
- циклы и условия;
- именованные URL и пространства имён;
- фильтры шаблонов;
- статические файлы;
- собственные CSS и JavaScript;
- текущую структуру веб-интерфейса.

Из раздела о планах развития необходимо удалить пункт о переходе на Django-шаблоны, поскольку после ПР6 эта задача считается выполненной.

При этом в `README.md` необходимо сохранить информацию о том, что:

- данные пока хранятся в JSON;
- Django ORM ещё не используется;
- формы ещё не реализованы;
- работа с базой данных относится к следующим практическим работам.

Пример файла `README.md` проекта Roomly после выполнения ПР6:

```
# Roomly – система бронирования помещений
Система бронирования помещений на Python с веб-интерфейсом на Django.
## 1. Назначение проекта
Roomly – учебное приложение для управления помещениями и их бронированием.
Проект позволяет хранить сведения о помещениях, пользователях и бронированиях, выполнять основные операции с этими объектами и предоставлять доступ к информации через веб-интерфейс Django.
Проект развивается поэтапно: консольная версия приложения дополняется веб-интерфейсом и постепенно переводится на возможности Django.
## 2. Используемые технологии
- Python 3;
- Django 5;
- Django Template Language (DTL);
- HTML5;
- CSS3;
- JavaScript;
- Bootstrap 5;
- JSON для хранения данных;
- Git;
- pytest;
- flake8.
## 3. Предметная область
В системе используются три основные сущности:
- Помещение (Room) – идентификатор, название и вместимость;
- Пользователь (User) – идентификатор и сведения о пользователе;
```

- Бронирование (Booking) – идентификатор, помещение, пользователь, дата бронирования и состояние бронирования.

Для предметной области реализованы собственные Python-классы.

На текущем этапе данные хранятся в JSON-файлах. Django ORM и база данных приложения пока не используются.

4. Структура проекта

```
room_booking/
├── bookings/
│   ├── templates/
│   │   └── bookings/
│   │       ├── includes/
│   │       │   └── booking_status.html
│   │       ├── booking_detail.html
│   │       └── booking_list.html
│   ├── urls.py
│   └── views.py
│
├── homepage/
│   ├── static/
│   │   └── homepage/
│   │       ├── css/
│   │       │   └── style.css
│   │       ├── img/
│   │       │   └── logo.png
│   │       └── js/
│   │           └── main.js
│   ├── templates/
│   │   └── homepage/
│   │       └── index.html
│   └── views.py
│
├── rooms/
│   ├── templates/
│   │   └── rooms/
│   │       ├── includes/
│   │       │   └── room_card.html
│   │       ├── room_detail.html
│   │       └── room_list.html
│   ├── urls.py
│   └── views.py
│
├── templates/
│   ├── includes/
│   │   └── navigation.html
│   └── base.html
│
├── models/
│   ├── bookings.py
│   ├── rooms.py
│   └── users.py
│
├── data/
│   ├── bookings.json
│   ├── rooms.json
│   └── users.json
│
├── tests/
│   ├── test_bookings.py
│   ├── test_rooms.py
│   └── test_users.py
│
├── roomly/
│   ├── settings.py
│   └── urls.py
│
├── main.py
├── storage.py
├── utils.py
├── manage.py
└── README.md
```

5. Веб-интерфейс
Веб-часть приложения реализована на Django.
Основные страницы:
- главная страница;
- список помещений;
- информация об отдельном помещении;
- список бронирований;
- информация об отдельном бронировании.
Для формирования страниц используются Django-шаблоны.
Генерация HTML непосредственно в Python-коде веб-представлений не используется. Функция page() больше не используется веб-представлениями.
Представления подготавливают данные и передают их в шаблоны через контекст.
6. Django Template Language
Шаблоны используют возможности Django Template Language.
Базовый шаблон
Общий шаблон страниц:
templates/base.html
Он содержит общую HTML-структуру страницы, подключение стилей и JavaScript, навигацию и блоки для содержимого отдельных страниц.
Дочерние шаблоны используют наследование:
{% extends "base.html" %}
Содержимое отдельных страниц размещается в блоке:
{% block content %}
{% endblock %}
Заголовок страницы задаётся через блок:
{% block title %}
{% endblock %}
Включаемые шаблоны
Повторяющиеся фрагменты вынесены в отдельные шаблоны.
Для помещений используется:
rooms/includes/room_card.html
Для отображения состояния бронирования:
bookings/includes/booking_status.html
Подключение выполняется с помощью:
{% include "rooms/includes/room_card.html" %}
или:
{% include "bookings/includes/booking_status.html" %}
Контекст
Представления передают данные в шаблоны через словарь контекста.
Например, для списка помещений:
```
context = {
    "rooms": rooms_list,
}
```
В шаблоне данные доступны по имени переменной:
{% for room in rooms %}
Для страницы отдельного помещения используется переменная:
{{ room }}
Аналогичный подход используется для бронирований.
Циклы и условия
Для формирования списков используются циклы:
{% for room in rooms %}
При отсутствии элементов используется:
{% empty %}
Условия используются для отображения различного содержимого страницы и состояния бронирования:
{% if booking.is_cancelled %}
7. Фильтры шаблонов
В шаблонах используются стандартные фильтры Django Template Language.
Например, количество помещений:
{{ rooms|length }}
Для отображения значения по умолчанию:
{{ room.name|default:"Без названия" }}
Фильтры используются для представления данных и не содержат бизнес-логику приложения.
Дата бронирования на текущем этапе хранится как строковое значение и выводится без преобразования средствами шаблона.
8. URL-маршрутизация
Для страниц используются именованные маршруты Django.
Примеры:
{% url 'rooms:list' %}
{% url 'rooms:detail' room.id %}

{% url 'bookings:bookings' %}
{% url 'bookings:detail' booking.id %}
Для приложений используются пространства имён (namespace), что позволяет обращаться к маршрутам через имена вида:
rooms:list
rooms:detail
bookings:bookings
bookings:detail
Ссылки в шаблонах формируются через {% url %}, а не за счёт жёстко заданных адресов.
9. Статические файлы
Статические файлы проекта находятся в каталоге:
homepage/static/homepage/
Используются:
css/style.css
js/main.js
img/logo.png
Для работы со статическими файлами в шаблонах используется:
{% load static %}
CSS подключается через:
{% static 'homepage/css/style.css' %}
Изображение:
{% static 'homepage/img/logo.png' %}
JavaScript:
{% static 'homepage/js/main.js' %}
В проекте используется собственный CSS для оформления элементов интерфейса.
JavaScript используется для небольшого динамического поведения интерфейса, в том числе для автоматического отображения текущего года в нижней части страницы.
10. Хранение данных
На текущем этапе данные приложения хранятся в JSON-файлах:
data/rooms.json
data/users.json
data/bookings.json
Загрузка данных выполняется средствами модуля storage.py.
Предметные объекты создаются на основе собственных Python-классов из каталога:
models/
Django ORM на текущем этапе не используется.
Каталоги и файлы, связанные с будущей работой с моделями Django, сохраняются для дальнейшего развития проекта.
11. Основные модули
storage.py
Отвечает за загрузку данных из JSON-файлов.
models/
Содержит классы предметной области:
- Room;
- User;
- Booking.
main.py
Содержит консольный вариант работы с приложением.
utils.py
Содержит вспомогательные функции для консольного интерфейса.
rooms/views.py
Обрабатывает веб-запросы, связанные с помещениями.
bookings/views.py
Обрабатывает веб-запросы, связанные с бронированиями.
homepage/views.py
Содержит представление главной страницы.
12. Тестирование
В каталоге tests/ находятся тесты основных классов предметной области:
tests/
├── test_bookings.py
├── test_rooms.py
└── test_users.py
Тесты проверяют работу основных операций с помещениями, пользователями и бронированиями.
Для запуска тестов используется:
pytest
13. Проверка качества кода
Для проверки соответствия кода требованиям форматирования и стиля используется flake8.
Запуск проверки:
flake8 . --extend-exclude=venv

```
## 14. Запуск проекта
Для запуска Django-сервера необходимо перейти в каталог проекта и выполнить:
python manage.py runserver
После запуска веб-интерфейс доступен по адресу:
http://127.0.0.1:8000/
## 15. Работа с Git
Исходный код проекта хранится в Git-репозитории.
В процессе разработки фиксируются изменения, связанные с отдельными этапами развития приложения.
Пример:
git status
git add .
git commit -m "PR6: Перевод веб-интерфейса на Django Templates"
## 16. Текущее состояние проекта
На текущем этапе Roomly содержит:
- предметные классы Room, User, Booking;
- консольный интерфейс;
- хранение данных в JSON;
- Django-проект;
- веб-представления для основных сущностей;
- Django Template Language;
- наследование шаблонов;
- базовый шаблон base.html;
- блоки шаблонов;
- включаемые шаблоны;
- циклы и условия;
- фильтры шаблонов;
- именованные URL и namespaces;
- статические CSS, JavaScript и графический файл;
- собственное оформление интерфейса;
- тесты предметной области.
Формы Django, модели Django ORM и работа с базой данных в текущей версии проекта не реализованы.
## 17. Дальнейшее развитие
Следующие этапы развития проекта предполагают переход от хранения данных в JSON к использованию базы данных
и Django ORM, а также реализацию форм Django и дальнейшее расширение веб-интерфейса.
В дальнейшем веб-приложение может быть дополнено операциями создания, изменения и удаления объектов, обра-
боткой пользовательского ввода, валидацией данных и API.
```

Разделы «Веб-интерфейс», «Django Template Language», «Фильтры шаблонов», «URL-маршрутизация» и «Статические файлы» описывают состояние проекта после ПР6. Информация о JSON-хранилище и отсутствии Django ORM и форм сохранена; переход на шаблоны в планах развития больше не упоминается как будущая задача.

5.13. Проверка и фиксация результата

5.13.1. Проверка запуска и всех страниц

Нужно убедиться, что после всех изменений проект работает. Запустите Django и проверьте отсутствие ошибок:

```
python manage.py runserver
```

Необходимо проверить следующие страницы:

URL	Что проверить
/	базовый шаблон, навигация, логотип, static-файлы, текущий год в подвале
/rooms/	цикл, карточки помещений, фильтр length
/rooms/<id>/	данные конкретного помещения, именованный URL
/rooms/9999/	страница «Помещение не найдено», статус 404
/bookings/	цикл, статус бронирования, дата
/bookings/<id>/	данные конкретного бронирования, навигация
/bookings/9999/	страница «Бронирование не найдено», статус 404

Дополнительно следует проверить:

- переход с главной страницы к помещениям;
- переход с главной страницы к бронированиям;
- переход из списка помещений к конкретному помещению;
- возврат из карточки помещения к списку;
- переход из списка бронирований к конкретному бронированию;
- возврат к списку;
- отображение пустой коллекции;
- отображение активного бронирования;
- отображение отменённого бронирования;
- отображение изображения;
- подключение CSS;
- выполнение JavaScript.

Для проверки поиска статических файлов можно выполнить:

```
python manage.py findstatic homepage/img/logo.png
```

Django должен найти файл в правильном каталоге.

5.13.2. Проверка перехода на шаблоны

После переноса страниц на шаблоны необходимо убедиться, что функция `page()` больше не используется веб-представлениями.

Необходимо проверить приложения `homepage`, `rooms` и `bookings` и убедиться, что view-функции используют:

```
render(request, ..., context)
```

Файлы HTML должны находиться в каталогах:

```
templates/
```

Поищите в веб-представлениях вызовы `page` и убедитесь, что функция больше не используется для формирования страниц. Теперь, когда весь проект переведён на `render()`, функцию `page()` в `homepage/views.py` можно удалить окончательно: её больше не импортируют `rooms/views.py` и `bookings/views.py`.

5.13.3. Проверка оформления шаблонов

В шаблонах необходимо проверить:

- единообразные отступы;
- понятные имена переменных;
- отсутствие дублирования общей HTML-структуры;
- использование `extends` для дочерних страниц;
- использование `block` для содержимого страниц;
- использование `include` для повторяющихся компонентов;

- использование `{% url %}` вместо жёстко заданных адресов;
- отсутствие Python-кода в HTML;
- отсутствие формирования HTML-строк во `view`;
- корректное закрытие тегов Django Template Language;
- корректную HTML-структуру.

5.13.4. Проверка проекта

Выполните:

```
python manage.py check
```

Запустите сервер:

```
python manage.py runserver
```

Проверьте страницы в браузере.

После этого выполните существующие тесты:

```
pytest -v
```

Проверьте стиль кода:

```
flake8 . --extend-exclude=venv
```

При наличии ошибок необходимо исправить их до создания итогового коммита.

5.13.5. Фиксация результата в Git

После завершения всех изменений:

```
git status
```

проверьте изменения:

```
git diff
```

добавьте файлы в индексируемые:

```
git add .
```

создайте коммит:

```
git commit -m "PR6: add Django templates and static files"
```

При использовании удалённого репозитория:

```
git push
```

Коммит ПР6 должен начинаться с префикса:

```
PR6:
```

6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

В результате выполнения ПР6 веб-часть индивидуального проекта должна быть переведена с формирования HTML в Python на систему шаблонов Django. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтвержден демонстрацией или содержимым репозитория.

Должны быть реализованы:

- базовый шаблон;
- наследование шаблонов;
- блоки `block`;
- включаемые шаблоны `include`;
- передача данных через контекст;
- циклы `{% for %}`;
- условные конструкции `{% if %}`;
- обработка пустых коллекций;
- именованные URL;
- пространства имён Django-приложений;
- ссылки через `{% url %}`;
- фильтры шаблонов;
- статические CSS-файлы;
- изображения;
- JavaScript;
- собственное оформление проекта;
- пять основных веб-страниц `Roomly`;
- отображение реальных данных предметной области;
- сохранение существующей JSON-модели проекта;
- отсутствие формирования HTML в Python через `page()`;
- обновлённый `README.md`;
- сохранённая работоспособность существующих тестов;
- Git-коммит с префиксом PR6:.

Архитектура веб-части проекта после выполнения работы показана на рис. 4. Источник данных не меняется: JSON-файлы и объекты `Room`, `User` и `Booking` загружаются функциями модуля `storage.py`.

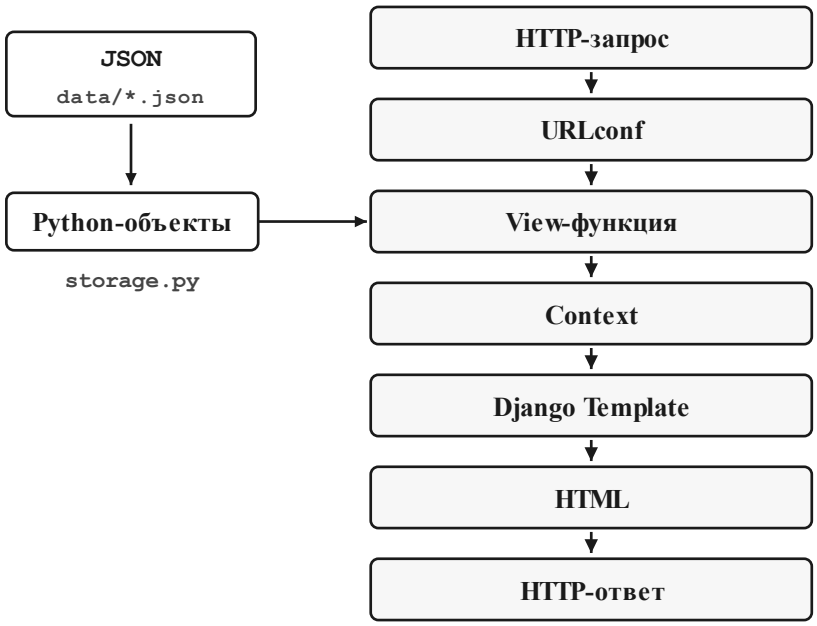


Рисунок 4 – Архитектура веб-части Roomly после ПР6

7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Защита практической работы проводится в форме демонстрации работающего проекта и устного ответа на контрольные вопросы. При защите необходимо продемонстрировать:

- запуск Django-проекта;
- работу главной страницы;
- работу страницы списка помещений;
- работу страницы отдельного помещения;
- работу списка бронирований;
- работу страницы отдельного бронирования;
- переходы между страницами;
- использование базового шаблона;
- наследование шаблонов;
- использование `include`;
- передачу данных из view в шаблон через контекст;
- использование циклов и условий;
- работу именованных URL и пространств имён;
- применение фильтров шаблонов;
- подключение собственного CSS;
- подключение изображения;
- подключение JavaScript;
- отсутствие формирования HTML через функцию `page()`;

- успешное выполнение `python manage.py check`;
- успешное выполнение тестов проекта;
- содержимое `README.md`;
- Git-коммит с префиксом PR6:.

7.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов раздела 04 «Основы веб-разработки». Отмечайте пункты по мере выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя.

№	Материал Яндекс Практикума	Выполнено
1	HTML и шаблоны Django / HTML в Django: шаблоны веб-страниц	☐
2	HTML и шаблоны Django / HTML-шаблоны из частей: теги include, extends, block	☐
3	HTML и шаблоны Django / Словарь контекста, ветвление и циклы в шаблонах	☐
4	HTML и шаблоны Django / Ссылки в шаблонах: name и namespace	☐
5	HTML и шаблоны Django / Фильтры в шаблонах: модифицируем контекст страницы	☐
6	HTML и шаблоны Django / Подключение стилей, картинок и скриптов к шаблонам	☐
7	HTML и шаблоны Django / Из тренажёра на компьютер. Настройка VS Code	☐
8	HTML и шаблоны Django / Правила оформления HTML и шаблонов Django	☐
9	HTML и шаблоны Django / Создание шаблонов из HTML-документов. Практика	☐
10	Финальный проект спринта Блогикум / Что предстоит сделать	☐
11	Финальный проект спринта Блогикум / Проект. Приложение «Блогикум». Часть 1	☐
12	Финальный проект спринта Блогикум / Обратная связь	☐

При защите студент должен быть готов подтвердить выполнение материалов курса и продемонстрировать результат соответствующих заданий в проекте.

7.2. Чек-лист выполнения индивидуального проекта

Чек-лист доработки индивидуального проекта:

№	Требование	Выполнено
1	Проект ПР1–ПР5 сохранён и продолжен	☐
2	Создана структура templates, настроен параметр DIRS	☐
3	Создан базовый шаблон base.html	☐
4	Используется {% extends %}	☐
5	Используются {% block %}	☐
6	Используется {% include %}	☐
7	Навигация вынесена в отдельный шаблон	☐
8	Созданы шаблоны для списка помещений	☐
9	Создан шаблон отдельного помещения	☐
10	Созданы шаблоны списка бронирований	☐
11	Создан шаблон отдельного бронирования	☐
12	Данные передаются через context	☐
13	Используется {% for %}	☐

№	Требование	Выполнено
14	Используется {% if %}	☐
15	Обрабатывается пустой список	☐
16	Настроены app_name	☐
17	Настроены именованные URL	☐
18	Используется {% url %}	☐
19	Используются пространства имён	☐
20	Используются фильтры шаблонов	☐
21	Подключён собственный CSS	☐
22	Подключено изображение	☐
23	Подключён JavaScript	☐
24	page() больше не используется веб-представлениями	☐
25	Все основные страницы работают	☐
26	Данные Roomly отображаются из существующей модели объектов	☐
27	JSON-хранилище ПР5 сохранено	☐
28	ORM не внедрялся преждевременно	☐
29	README.md обновлён	☐
30	python manage.py check выполнен без ошибок	☐
31	pytest -v выполнен	☐
32	flake8 выполнен	☐
33	Создан Git-коммит с префиксом PR6:	☐

7.3. Что необходимо показать при защите

На компьютере необходимо открыть проект и последовательно продемонстрировать:

1. Базовый шаблон – показать `templates/base.html`, объяснить назначение блоков:

```
{% block title %}...{% endblock %}
{% block content %}...{% endblock %}
```

2. Наследование – показать один из дочерних шаблонов (`rooms/templates/rooms/room_list.html`) и строку:

```
{% extends "base.html" %}
```

3. Включаемый шаблон – показать `rooms/templates/rooms/includes/room_card.html` и его подключение:

```
{% include "rooms/includes/room_card.html" %}
```

4. Контекст – показать соответствующую view-функцию и шаблон. Например:

```
context = {
    "rooms": rooms_list,
}
{% for room in rooms %}
```

5. URL namespace – показать `rooms/urls.py`:

```
app_name = "rooms"
```

и ссылку:

```
{% url "rooms:detail" room.id %}
```

6. `static` – показать `{% load static %}` и подключение собственного CSS:

```
{% static "homepage/css/style.css" %}
```

а также каталог:

```
homepage/static/homepage/
```

7.4. Критерии оценки

Практическая работа считается выполненной при соблюдении следующих условий.

№	Критерий	Содержание
1	Материалы Практикума	Выполнены предусмотренные материалы раздела 04
2	Запуск проекта	Проект запускается без ошибок; все основные страницы открываются
3	Шаблоны	HTML вынесен из Python-кода в шаблоны; используется base.html; применяются extends и block
4	Включаемые шаблоны	Повторяющиеся фрагменты вынесены через include
5	Контекст	Данные передаются через контекст; используются циклы и условия
6	Именованные URL	Ссылки формируются через именованные URL; используются пространства имён
7	Фильтры	Применяются фильтры шаблонов
8	Статические файлы	Подключены собственные статические файлы проекта: CSS, изображение, JavaScript
9	Данные предметной области	Данные Roomly отображаются корректно из существующей модели объектов
10	README	Документация обновлена и соответствует текущему состоянию проекта
11	Работоспособность	Функциональность ПР3–ПР5 не нарушена; выполнены проверки проекта: python manage.py check, pytest, flake8
12	Git	Изменения зафиксированы в репозитории коммитом с меткой PR6:

7.5. Требования к отчету

Объем отчета – 1–2 страницы, не считая титульной информации. Отдельное подробное описание каждого упражнения Яндекс Практикума не требуется. Отчет должен содержать:

1. Общие сведения:

- дисциплину;
- номер практической работы;
- тему практической работы;
- учебную группу;
- ФИО студента;
- вариант.

2. Цель работы: кратко указать цель освоения шаблонов Django и их применения в индивидуальном проекте.

3. Выполненные материалы Яндекс Практикума:

Привести перечень выполненных материалов ПР6 с отметкой о выполнении:

Раздел	Выполненные темы
Раздел 04. «Основы веб-разработки»	Тема «HTML и шаблоны Django»: шаблоны веб-страниц, словарь контекста, ветвление и циклы, именованные URL и пространства имён, фильтры, подключение стилей, картинок и скриптов
Раздел 04. «Основы веб-разработки»	Тема «Финальный проект спринта Блогикум»

4. Доработка индивидуального проекта. Указать:

- созданную структуру шаблонов;
- базовый шаблон;
- наследование;
- include;
- передачу данных через контекст;
- циклы и условия;
- именованные URL;
- фильтры;
- статические файлы.

5. Результат работы: указать реализованные страницы и основные изменения веб-интерфейса.

6. Проверка. Привести результаты:

```
python manage.py check
pytest -v
flake8 . --extend-exclude=venv
```

7. Git: указать созданный коммит ПР6.

8. Вывод: сделать краткий вывод о результатах выполнения практической работы.

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите необходимо уметь ответить на контрольные вопросы и продемонстрировать соответствующий участок проекта.

Шаблоны Django и контекст

1. Для чего используются шаблоны Django?
2. Почему формирование HTML-кода непосредственно в Python считается неудобным при развитии Django-приложения?
3. Для чего используется функция render()?
4. Что такое контекст шаблона?

5. Как передать из view в шаблон список объектов?

6. Для чего используются конструкции {{ ... }}?

7. Чем {% if %} отличается от {% for %}?

8. Для чего используется {% empty %}?

Наследование и включение шаблонов

9. Что такое базовый шаблон?

10. Для чего используются {% extends %} и {% block %}?

11. Для чего используется {% include %}?

12. В каких случаях целесообразно выносить часть HTML в отдельный шаблон?

Именованные URL и пространства имён

13. Для чего используется параметр name в path()?

14. Что такое app_name в urls.py?

15. Зачем используются пространства имён URL?

16. Чем отличается ссылка /rooms/ от {% url "rooms:list" %} с точки зрения организации Django-приложения?

Фильтры шаблонов

17. Для чего используются фильтры шаблонов?

18. Что делает фильтр length?

19. Для чего используется фильтр date?

Статические файлы

20. Для чего нужен тег {% load static %}?

21. Где должны храниться статические файлы Django-приложения?

22. Как подключить CSS-файл через систему static files?

23. Как подключить изображение к шаблону?

24. Как подключить JavaScript-файл?

25. Почему собственный CSS проекта не следует полностью заменять подключением Bootstrap?

Архитектура и индивидуальный проект

26. Какие задачи должны выполняться во view, а какие – в шаблоне?

27. Почему бизнес-логику не следует переносить в Django-шаблоны?

28. Почему в ПР6 данные по-прежнему загружаются из JSON, а не из Django ORM?

29. Чем архитектура веб-части проекта после ПР6 отличается от архитектуры ПР5?

30. Для чего в ПР6 необходимо отказаться от функции page()?

9. ИТОГ

После выполнения ПР6 проект переходит от формирования HTML-страниц непосредственно в Python-коде к стандартной для Django организации представления данных:

```
View → Context → Template → HTML
```

При этом существующая предметная модель проекта сохраняется (`Room`, `User`, `Booking`), а источник данных на данном этапе остаётся прежним – JSON-файлы.

Следующим этапом развития проекта станет переход от JSON-хранилища к реляционной базе данных и Django ORM.

Таким образом, ПР6 завершает работу с шаблонным уровнем Django и подготавливает индивидуальный проект к дальнейшему переходу (рис. 5):

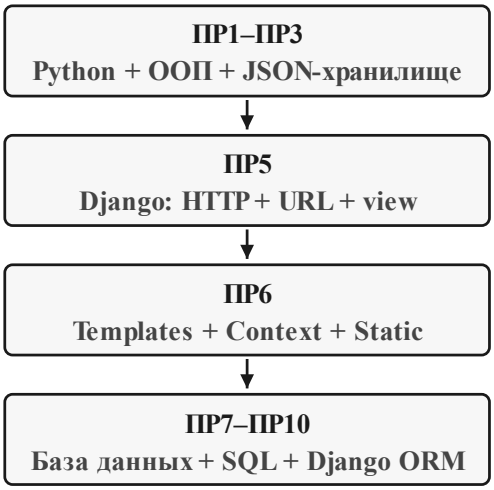


Рисунок 5 – Развитие индивидуального проекта